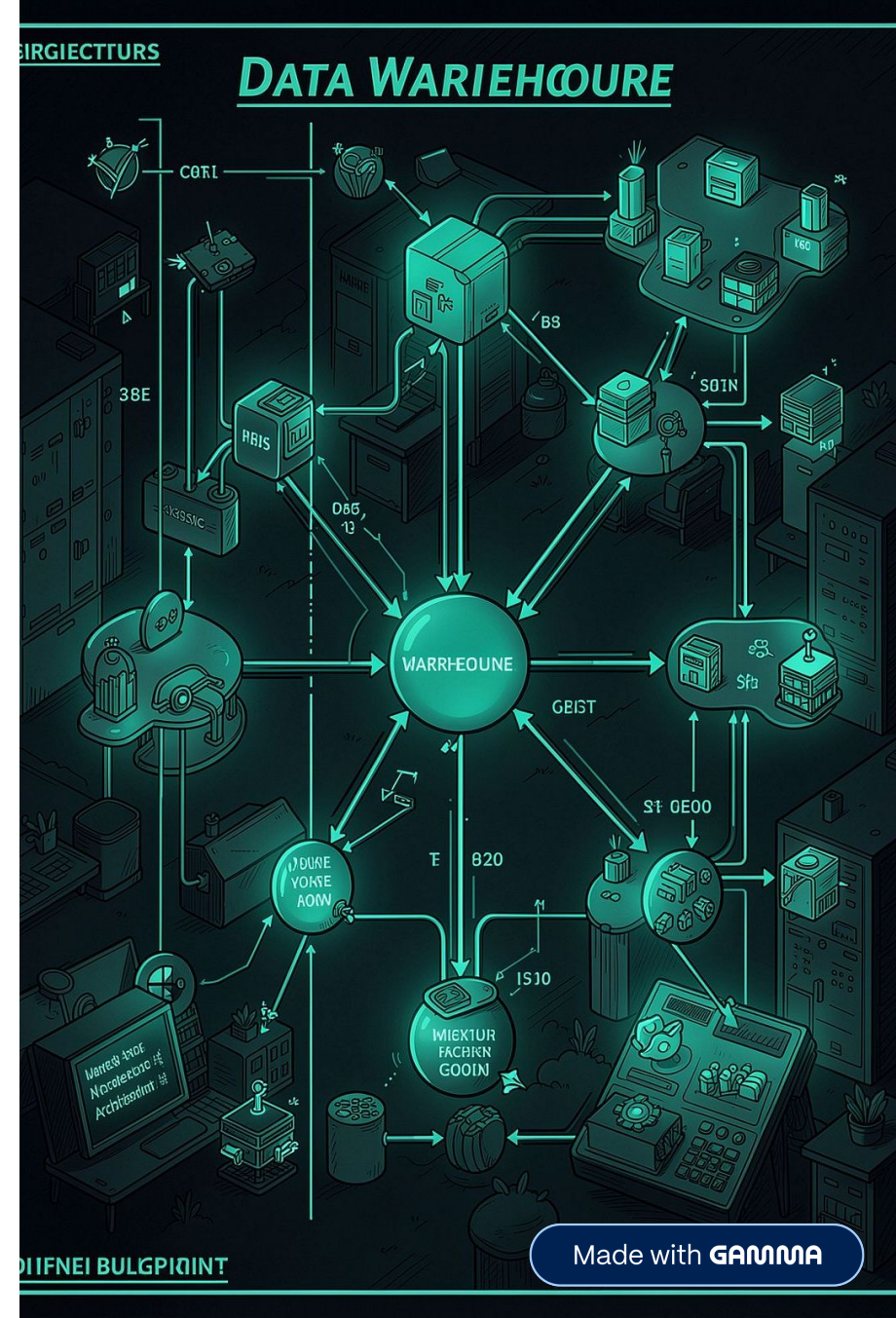


Data Warehouse Kimball

Ente Certificatore Privato - Progettazione dimensionale, ETL con Apache NiFi e reportistica su PostgreSQL



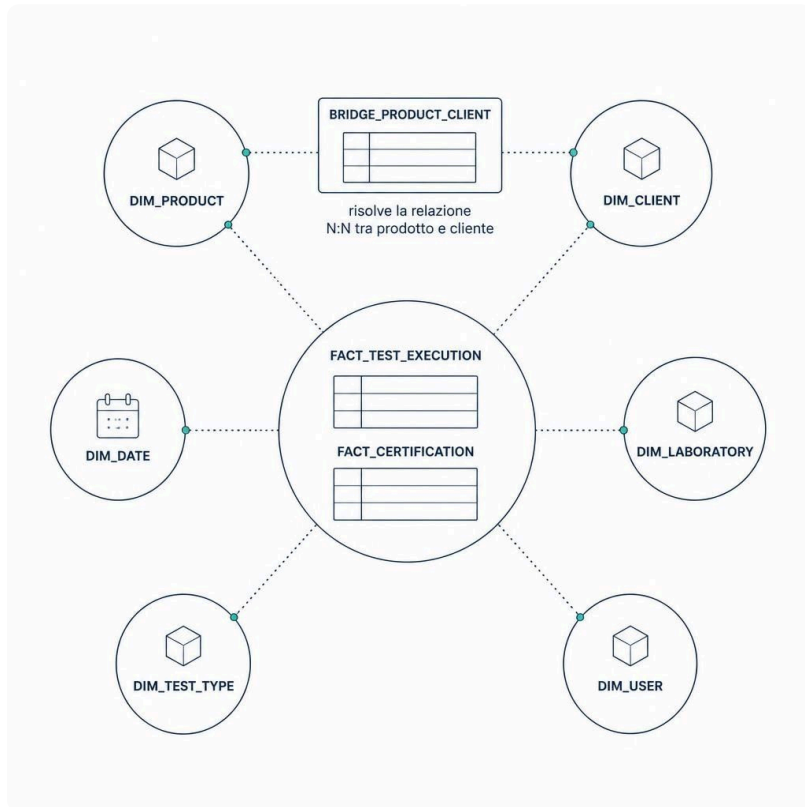
Analisi e Progettazione Dimensionale

Modello OLTP sorgente

14 entità: APP_USER, ARCHIVE, CATEGORY, CERTIFICATION, CLIENT, INPUT_MARKET, LABORATORY, LABORATORY_TEST, PEOPLEGROUP, PRODUCT, PRODUCT_CLIENT, PRODUCT_COMPONENTS, TEST_CATEGORY, USER_GROUP.

Schema Kimball progettato

- **2 Fact Table:** FACT_TEST_EXECUTION (grain: un test eseguito in laboratorio) e FACT_CERTIFICATION (grain: una certificazione emessa) → per evitare fan-out
- **6 Dimensioni:** DIM_CLIENT, DIM_DATE, DIM_LABORATORY, DIM_PRODUCT, DIM_TEST_TYPE, DIM_USER
- **SCD Tipo 2** su DIM_CLIENT (storicizza email/ragione sociale)
- **Bridge table:** BRIDGE_PRODUCT_CLIENT per risolvere la relazione N:N Prodotto<->Cliente senza sporcare il grano delle fact table.



Ambiente e Creazione Tabelle

Ambiente

PostgreSQL 18 + Apache NiFi nativi su Windows.

Database **certification_project** con due schema:

- **source_system** (14 tabelle OLTP)
- **dw** (10 tabelle Kimball)

Creazione Tabelle

- **category** creata a mano via pgAdmin (esercizio su PK/Check Constraint).
- **13 tabelle** di source_system e **10 tabelle** dw via script SQL in ordine di dipendenza FK

Strategia Ibrida di Popolamento

Anagrafiche Piccole → Excel + CSV

Valori di dominio reali e finiti, curati a mano.

Tecnica di coerenza: riferimenti alla stessa riga del foglio

"Liste" per evitare combinazioni incoerenti.

- **category:** 7 righe (Footwear, Toys, Electronics...)
- **laboratory:** 9 righe (uno per paese partner)
- **input_market:** 6 righe (MKT-EUR, MKT-NA, MKT-SA...)
- **user_group:** 7 righe (ruoli aziendali)

Totale: 974.387 Record Generati

Tabelle a Volume Alto → SQL Nativo

`generate_series` + `random()`. NiFi riservato a infasamento e incrementale.

- **`generate_series(1,N)`:** righe contatore
- **`random()` + `floor()`:** FK casuali e valori da array
- **`ON CONFLICT DO NOTHING`:** elimina duplicati su chiavi composte
- **`lpad()`:** codici progressivi (es. PRD-0000001)

Infasamento con Apache NiFi

1. GenerateFlowFile

Crea un flowfile "grilletto" vuoto. Scheduling Timer Driven per l'incrementale automatico.

2. ReplaceText

Inietta la query SQL di trasformazione (INSERT INTO dw.xxx SELECT... FROM source_system) nel contenuto del flowfile.

3. PutSQL

Esegue la query su Postgres via DBCPConnectionPool. NiFi è orchestratore, non transita le righe fisicamente.

9 Tabelle DW Popolate in Ordine

Tabella DW	Origine / Logica	Righe
dim_date	generate_series 2018–2027	3.652
dim_product	product + category (denormalizzata)	100.000
dim_client	SCD2 init: valid_from='2018-01-01', is_current=TRUE	5.500
dim_laboratory	mapping diretto da source	9
dim_test_type	test_category, has_attachment calcolato	170.000
dim_user	JOIN peplegroup + user_group (LIMIT 1)	13.500
bridge_product_client	chiavi naturali → surrogate	~119.992
fact_test_execution	JOIN su 4 dimensioni collegate	190.000
fact_certification	lead_time_days = cert_date – creation_date	100.000

Caricamento Incrementale

Meccanismo Delta

- Rilevamento tramite colonna **updated_at** in source_system
- Watermark salvato in **dw.etl_load_control** (una riga per flow_name)
- Condizione: **WHERE updated_at > watermark AND NOT EXISTS**

SCD Tipo 2 su dim_client

Due passaggi: UPDATE per chiudere versione storica (valid_to=oggi, is_current=FALSE) + INSERT nuova riga (valid_from=oggi, is_current=TRUE).

Risultati test

- dim_test_type: 170.000 → 170.200
- fact_test_execution: 190.000 → 190.200
- fact_certification: 100.000 → 100.080

Automazione e Architettura ELT

Automazione reale

Query riscritte come blocchi **PL/pgSQL autonomi** (DO \$\$...\$\$): leggono watermark, eseguono delta, aggiornano watermark in un'unica transazione. 8 Process Group con Timer Driven (1 min per demo) - esecuzione continua senza intervento manuale.

Copertura completa

5 flussi iniziali + 3 aggiuntivi (dim_laboratory, dim_user, bridge_product_client). Test end-to-end superato: INSERT/UPDATE su source rilevati automaticamente in dw senza intervento manuale.

Limite documentato

I DELETE da source_system **non** vengono intercettati (solo INSERT/UPDATE via updated_at). Scelta consapevole: comune nei DW reali per preservare lo storico analitico.

Query

Costrutti SQL coperti

INNER / LEFT / RIGHT
JOIN

FULL OUTER JOIN

GROUP BY + HAVING

CTE + Subquery

Window Functions

Query di business sviluppate

- Query base
- Query di Business
- Query di Reportistica

📄 **Limite di perimetro:** input_market e product_components non modellati nello star schema — conseguenza consapevole delle decisioni di Fase 2 (bus matrix limitata ai processi scelti).

Correzione: user_group_name NULL

Problema

~3.286 utenti su 13.500 (24%) con user_group_name = NULL in dim_user. Causa: la generazione casuale di peoplegroup non garantiva copertura statistica di ogni utente.

Soluzione

1. INSERT mirato in source_system.peoplegroup per utenti privi di associazione → copertura 100%
2. UPDATE di backfill su dw.dim_user → 0 utenti con NULL residui

Dati Sensibili, Errori e Considerazioni Critiche



Dati Sensibili

Individuati: user_password, client_email, full_name, user_location. Da migliorare attraverso hashing robusto (bcrypt)



Gestione Errori NiFi

Corretto su tutti gli 8 Process Group: aggiunto LogAttribute (Log Level = error) sulla relazione "failure" di PutSQL — prima i fallimenti venivano scartati silenziosamente.



NiFi "Sotto-utilizzato"?

Sì ma con valore reale: monitoraggio visivo, data provenance, gestione errori strutturata, estensibilità futura a fonti eterogenee.